

Herald



Herald — commons_workable Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-30
Last modified	2026-05-30
Status	active
Status summary	Nano-detail per-module reference for <code>commons_workable</code> — the SQLite workable-items single-source-of-truth (SSoT) store that Herald shares with ATMOSphere. Documents the schema (<code>items / item_history / meta</code>), the 10-value status closed set, the <code>Store + Repo</code> CRUD API, the <code>Diff</code> change-feed (5 Change Kinds), and <code>ParseTracker</code> (ATMOSphere's real <code>## §...</code> [ATM-NNN] Markdown format). ANTI-BLUFF: every section documents only what the source under <code>commons_workable/</code> actually does as of this revision; not-yet-built call sites are marked <code>PLANNED</code> and never implied to work.
Issues	(none specific to this guide)
Continuation	bump when the DB is materialized at the canonical <code>docs / workable_items.db</code> path (HRD-131 / HRD-155), when <code>item_history</code> and <code>meta</code> gain a typed Go API beyond raw <code>Store.DB()</code> , and when the bidirectional MD ² SQLite regenerator lands (HRD-150).

Table of contents

- [§1. Overview](#)
 - [§2. The schema](#)
 - [§3. The Store API](#)
 - [§4. The Repo CRUD API](#)
 - [§5. The Diff change-feed](#)
 - [§6. ParseTracker — the Markdown reader](#)
 - [§7. Pure-Go SQLite — why no CGO](#)
 - [§8. Usage examples](#)
 - [§9. Testing notes](#)
 - [§10. References](#)
-

§1. Overview

`commons_workable` (Go package `workable`, module path `github.com/vasic-digital/herald/commons_workable`) is the keystone of the ATMOSphere[?]Herald workable-items integration. It mirrors ATMOSphere's workable-items SQLite SSOT schema **verbatim** so that a database file created by either project is interchangeable, and provides four building blocks:

Building block	Source file	Responsibility
Store	<code>store.go</code>	Schema-applying connection holder (WAL + foreign-keys pragmas).
Repo	<code>crud.go</code>	CRUD repository over the <code>items</code> table with status validation.
Item/ StatusValues	<code>item.go</code>	The row mapping + the 10-value status closed set.
Diff/Change	<code>changefeed.go</code>	Per-property change feed between two item snapshots.
ParseTracker	<code>parser.go</code>	Tolerant parser for ATMOSphere's real Markdown tracker format.

The package doc (top of `store.go`) states the contract directly: it "mirrors ATMOSphere's workable-items SQLite SSOT so that Herald and ATMOSphere can share one database file."

This module sits at the L0/commons layer and is consumed by the watch + workflow bridge (see `commons_watch`, `docs/guides/COMMONS_WATCH.md`, and `docs/guides/WORKABLE_ITEMS_INTEGRATION.md`).

Path note (PLANNED reconciliation). Per Helix §11.4.95, the constitution's canonical on-disk path for the committed DB is `docs/workable_items.db` (no leading dot). Materialization of the live committed DB at that path is tracked work (HRD-131 / HRD-155) and is not performed by this module — `commons_workable` only opens whatever path a caller hands `Open()`.

§2. The schema

The canonical DDL is the `schemaDDL` constant in `store.go`, applied idempotently (`CREATE TABLE IF NOT EXISTS`) on every `Open()`. It defines three tables.

§2.1 items — the workable-item rows

Column	Type / constraint	Notes
<code>atm_id</code>	TEXT NOT NULL	e.g. ATM-238. Part of the composite PK.
<code>type</code>	TEXT CHECK (type IN ('Bug', 'Feature', 'Task'))	May be "" for a non-item heading the parser skips.
<code>status</code>	TEXT	One of the 10 status values (see §2.4); validated in Go, not by a DB CHECK.

Column	Type / constraint	Notes
severity	TEXT	Free-text severity label.
title	TEXT	Human-readable title.
description	TEXT	Description prose.
forensic_anchor	TEXT	The §11.4-style forensic anchor.
closure_criteria	TEXT	Closure criteria text.
composes_with	TEXT	A JSON array stored as text, e.g. <code>["ATM-100"]</code> .
current_location	TEXT CHECK (current_location IN ('Issues', 'Fixed')) DEFAULT 'Issues'	Part of the composite PK.
body_md	TEXT	The raw Markdown body block under the item heading.
created_at	TEXT	ISO date string.
last_modified	TEXT	ISO date string.

Composite primary key: PRIMARY KEY (atm_id, current_location). A single ATM-NNN id can therefore exist once in `Issues` and once in `Fixed` simultaneously — the location is part of the identity, which is what makes the §11.4.19 atomic `Issues`→`Fixed` migration representable as two rows rather than one mutable row.

§2.2 item_history — the lifecycle event log

Column	Type / constraint
id	INTEGER PRIMARY KEY AUTOINCREMENT
atm_id	TEXT
event_type	TEXT CHECK (event_type IN ('Opened', 'Updated', 'Reopened', 'Fixed', 'Implemented', 'Completed'))
by	TEXT CHECK (by IN ('AI', 'User'))
on_date	TEXT
reason	TEXT
evidence_path	TEXT
created_at	TEXT

This table is created by `Open()` but **has no typed Go API in this module today**. Callers that need to write or read history use the raw `*sql.DB` exposed by `Store.DB()` (see §3). A typed history API is PLANNED.

§2.3 meta — key/value metadata

Column	Type / constraint
key	TEXT PRIMARY KEY
value	TEXT
last_modified	TEXT

Also created by `Open()`; accessed via `Store.DB()` only (no typed API yet).

§2.4 The 10-value status closed set

`StatusValues(item.go)` is the canonical closed set. `Create` and `Update` reject any status outside it before any DB access.

Status value

- 1 Queued
- 2 In progress
- 3 Ready for testing
- 4 In testing
- 5 Reopened
- 6 Operator-blocked
- 7 Fixed ($\rightarrow$ Fixed.md)
- 8 Implemented ($\rightarrow$ Fixed.md)
- 9 Completed ($\rightarrow$ Fixed.md)
- 10 Obsolete ($\rightarrow$ Fixed.md)

`ValidStatus(status string) bool` reports membership; it is backed by a precomputed `statusSet` map for $O(1)$ lookup. Note the four terminal ($\rightarrow$ Fixed.md) values (7–10) correspond to the four `current_location='Fixed'` outcomes.

§2.5 WAL + foreign keys

`Open()` issues `PRAGMA journal_mode=WAL` and `PRAGMA foreign_keys=ON`, and pins the pool to a single connection (`db.SetMaxOpenConns(1)`) so those connection-scoped pragmas hold for every query the `Store` issues. WAL is the mode that makes the `commons_watch` WAL-poll fallback necessary — see `docs/guides/COMMONS_WATCH.md` §3.

§3. The Store API

`Store(store.go)` holds one open `*sql.DB` with the schema applied.

Function / method	Signature	Behaviour
Open	<code>Open(path string) (*Store, error)</code>	Opens (creating if absent) the SQLite DB at <code>path</code> , pins to a single connection, enables WAL + foreign keys, then applies <code>schemaDDL</code> idempotently. Wraps every failure with a <code>workable: prefix</code> and closes the half-open handle on error.
<code>(*Store)</code> DB	<code>DB() *sql.DB</code>	Exposes the underlying <code>*sql.DB</code> for raw access — <code>item_history</code> inserts, <code>meta</code> reads, and tests.
<code>(*Store)</code> Close	<code>Close() error</code>	Releases the underlying connection pool.

`Open` is safe to call repeatedly against the same file — schema application is idempotent (verified by `TestOpen_CreatesSchemaIdempotently`).

§4. The Repo CRUD API

Repo (crud.go) is a CRUD repository over the items table. Construct it with NewRepo(s *Store) *Repo. All methods take a context.Context.

Method	Signature	Behaviour
Create	Create(ctx, it Item) error	Validates it.Status against the closed set first (rejects unknown/empty loudly with invalid status ...), then INSERTs all 13 columns.
GetByID	GetByID(ctx, atmID, location string) (*Item, error)	Selects by the composite key. Returns (nil, nil) when no such row exists (absence is not an error here).
Update	Update(ctx, it Item) error	Validates status first, then UPDATEs all mutable columns (everything except the PK pair) WHERE atm_id=? AND current_location=?. A zero-rows update is reported as ErrNotFound.
Delete	Delete(ctx, atmID, location string) error	Deletes by composite key. A zero-rows delete is reported as ErrNotFound.
List	List(ctx, location string) ([]Item, error)	Returns every item at location, ORDER BY atm_id for deterministic output.

§4.1 ErrNotFound and "loud on missing"

ErrNotFound = errors.New("workable: item not found"). Update and Delete both route through the internal requireOneRow helper, which inspects RowsAffected() and returns a wrapped ErrNotFound (%w, so errors.Is(err, ErrNotFound) works) when zero rows matched. This is the deliberate "loud on missing" contract — a no-op update/delete is treated as a caller error, not silently swallowed.

GetByID is the deliberate exception: a missing row there is the normal "not present yet" signal and returns (nil, nil).

§4.2 Status validation boundary

Status validation happens in Go (ValidStatus) **before** any DB access, on both Create and Update. The items.status column itself has no SQL CHECK, so the Go layer is the sole enforcement point for the 10-value closed set — keep all writes flowing through Repo rather than raw Store.DB() to preserve that guarantee.

§5. The Diff change-feed

Diff(prev, curr []Item) []Change (changeFeed.go) computes the deterministic per-property delta transforming prev into curr. Items are keyed on their composite (atm_id, current_location).

§5.1 The 5 Change Kinds

Kind constant	Value	Emitted when	Field / Old / New
KindCreated	<code>item.created</code>	key present in curr, absent in prev	empty (lifecycle)
KindDeleted	<code>item.deleted</code>	key present in prev, absent in curr	empty (lifecycle)
KindStatusChanged	<code>item.status.changed</code>	status differs	Field="status", Old→New
KindFieldChanged	<code>item.field.changed</code>	severity, title, or type differs	one Change each, Field named
KindContentUpdated	<code>item.content.updated</code>	body_md or description differs	one Change each, Field named

A Change carries AtmID, Location, Kind, Field, Old, New. For the two lifecycle Kinds, Field/Old/New are empty.

§5.2 Worked examples

```
prev: ATM-1 {status:"Queued"}                      curr: (absent)
-> Change{AtmID:"ATM-1", Location:"Issues", Kind:"item.deleted"}
```

```
prev: (absent)                      curr: ATM-1 {status:"Queued"}
-> Change{AtmID:"ATM-1", Location:"Issues", Kind:"item.created"}
```

```
prev: ATM-1 {status:"Queued"}                      curr: ATM-1 {status:"InProgress"}
-> Change{Kind:"item.status.changed", Field:"status", Old:"Queued", New:"InProgress"}
```

```
prev: ATM-1 {severity:"Low", title:"old", type:"Bug"}
curr: ATM-1 {severity:"High", title:"new", type:"Feature"}
-> item.field.changed Field:"severity" Old:"Low" New:"High"
-> item.field.changed Field:"title" Old:"old" New:"new"
-> item.field.changed Field:"type" Old:"Bug" New:"Feature"
```

```
prev: ATM-1 {body_md:"old-body", description:"old-desc"}
curr: ATM-1 {body_md:"new-body", description:"new-desc"}
-> item.content.updated Field:"body_md" Old:"old-body" New:"new-body"
-> item.content.updated Field:"description" Old:"old-desc" New:"new-desc"
```

§5.3 Deterministic ordering

The output slice is sorted (stable) by:

1. AtmID ascending, then
2. Location ascending, then
3. **Kind-group rank** — created(0) → deleted(1) → status.changed(2) → field.changed(3) → content.updated(4), then
4. Field name ascending.

Within one item this means a `field.changed` burst always emits in `severity`, `title`, `type` order (alphabetical by field), and a `content.updated` burst in `body_md`, `description` order — confirmed by `TestDiff_FieldChanged_TitleSeverityType` and `TestDiff_ContentUpdated`. The full ordering is fully deterministic regardless of map iteration order, which is what lets downstream notification formatting be reproducible.

Scope note. `Diff` only inspects the tracked properties listed above (`status`, `severity`, `title`, `type`, `body_md`, `description`). Changes to other `Item` fields (`forensic_anchor`, `closure_criteria`, `composes_with`, `created_at`, `last_modified`) do **not** currently produce a `Change`. Treat that as the documented current behaviour, not an oversight to assume around.

§6. ParseTracker — the Markdown reader

`ParseTracker(markdown, location string) ([]Item, error)`
(`parser.go`) parses ATMOSphere's real Markdown tracker format and returns the workable items it finds, each tagged with the supplied location ("Issues" or "Fixed").

§6.1 Heading shapes recognised

Items are H2 headings (`## ...`). The parser handles these shapes:

Heading shape	Treatment
<code>## §GL CRITICAL — [ATM-238] Netflix login failure on D3</code>	item; id ATM-238, title Netflix login failure on D3
<code>## SYS — [ATM-101] Disk pressure alerting</code>	item; id ATM-101, title Disk pressure alerting
<code>## §UX — Tidy the onboarding copy</code>	item with no bracket id → id is derived (see §6.3)
<code>## A. Global blockers</code>	section header — skipped (see §6.2)

`atmIDRe (\ [(ATM-\d+)\ ])` extracts the bracket id. `headingTitle` strips the prefix segment by splitting on the em-dash separator — (or -, or a leading A. section-style dot within the first 3 chars), then removes any surviving `[ATM-NNN]` bracket and trims.

§6.2 Section-header skipping

The decisive test is the metadata block, not the heading text. For each H2 block the parser captures the body lines up to (but excluding) the next H2 and runs `scanMeta` over them. `scanMeta` matches lines of the form `**Status:** value`, `**Type:** value`, `**Severity:** value` (case-insensitive, via `metaRe`).

A heading whose body contains no `**Status:**` line is treated as a section header and skipped (the `if status == "" { continue }` guard). So `## A. Global blockers`, which has only prose under it, never becomes an `Item` — confirmed by `TestParseTracker_SectionHeaderSkipped`.

§6.3 Derived stable ids

A heading carrying a [ATM-NNN] bracket takes that as its id. A heading **without** one gets a stable, deterministic id: `deriveID` computes `sha1(heading text)` and returns "ATM-DERIVED-" + first 8 hex chars. Parsing the same heading text again yields the same id (verified by `TestParseTracker_DerivesStableIDWhenNoBracket`), so a bracket-less item keeps a stable identity across reparse.

§6.4 What ParseTracker populates

A parsed `Item` gets `AtmID`, `Type`, `Status`, `Severity`, `Title`, `CurrentLocation` (from the `location` argument), and `BodyMd` (the raw body block, right-trimmed of trailing newlines, never bleeding into the next heading — see the body-isolation assertion in `TestParseTracker_RepresentativeItem`). The remaining `Item` fields (`Description`, `ForensicAnchor`, `ClosureCriteria`, `ComposesWith`, `CreatedAt`, `LastModified`) are **left zero** by the parser; they are populated (if at all) by other layers, not here.

§7. Pure-Go SQLite — why no CGO

The module imports `modernc.org/sqlite` (registered as the "sqlite" driver via a blank import in `store.go`), the **pure-Go** SQLite implementation — **not** the CGO `mattn/go-sqlite3`. The `go.mod` module doc states the rationale directly: "It uses the PURE-GO `modernc.org/sqlite` driver (no CGO) so it builds and tests anywhere without a C toolchain."

This is an anti-bluff (§107 / Helix §11.4) choice with concrete payoff:

- **In-process tests need no container.** Every test opens a real on-disk DB under `t.TempDir()` and exercises real SQL — no Postgres/Redis container, no `CGO_ENABLED=1`, no C compiler. The PASS is genuine end-to-end SQLite behaviour, not a mock.
- **Cross-platform reproducibility.** `go build/go test` work on any host the Go toolchain supports without platform-specific C build steps.
- **Single-binary flavors.** The flavor binaries that depend on this module stay statically linkable without CGO.

§8. Usage examples

§8.1 Open, create, read, list

```
import workable "github.com/vasic-digital/herald/
    commons_workable"

s, err := workable.Open("/path/to/workable_items.db")
if err != nil { /* handle */ }
defer s.Close()

repo := workable.NewRepo(s)
ctx := context.Background()
```



```

err = repo.Create(ctx, workable.Item{
    AtmID:      "ATM-238",
    Type:       "Bug",
    Status:     "Operator-blocked", // MUST be in
                StatusValues
    Severity:   "Critical",
    Title:      "Netflix login failure on D3",
    CurrentLocation: "Issues",
    ComposesWith: `["ATM-100"]`,      // JSON array as text
})

it, err := repo.GetByID(ctx, "ATM-238",
    "Issues") // (nil,nil) if absent
items, err := repo.List(ctx, "Issues")           // ordered by
    atm_id

```

§8.2 Parse a tracker and diff against the DB snapshot

```

parsed, _ := workable.ParseTracker(markdownText, "Issues")

prev, _ := repo.List(ctx, "Issues")
changes := workable.Diff(prev, parsed)
for _, c := range changes {
    switch c.Kind {
    case workable.KindStatusChanged:
        // c.Field == "status", c.Old -> c.New
    case workable.KindCreated, workable.KindDeleted:
        // lifecycle: c.AtmID / c.Location only
    }
}

```

§8.3 Detecting "not found"

```

if err := repo.Update(ctx, it); errors.Is(err,
    workable.ErrNotFound) {
    // no row matched (it.AtmID, it.CurrentLocation)
}

```

§9. Testing notes

All tests live alongside the source in `commons_workable/` and run with no external services (pure-Go SQLite, §7):

```
go test -race -count=1 ./commons_workable/...
```

Test file	Covers
<code>store_test.go</code>	Open creates all 3 tables, WAL active, foreign_keys=1, idempotent re-open.
<code>crud_test.go</code>	

Test file	Covers
	Full CRUD round-trip; (nil, nil) on absent GetByID; Create rejects unknown + empty status; Update/Delete loud-on-missing.
<code>changefeed_test.go</code>	Each of the 5 Kinds; field-order determinism; cross-item <code>atm_id</code> ordering; no-change → empty.
<code>parser_test.go</code>	Representative real-format tracker; plain-prefix item; section-header skip; derived-id stability; exact item count (3 from the fixture).

Anti-bluff observations worth preserving when editing tests:

- `crud_test.go` round-trip asserts `*got == in` (whole-struct equality), so a dropped column would fail loudly.
- `parser_test.go` asserts `body_md` does **not** bleed into the next heading — the body-isolation invariant.
- `changefeed_test.go` uses `reflect.DeepEqual` on the full `[]Change`, so any ordering or field drift fails.

§10. References

- Source: `commons_workable/{store.go, item.go, crud.go, changefeed.go, parser.go}` and their `_test.go` siblings.
- Module doc: `commons_workable/go.mod` header comment (pure-Go SQLite rationale).
- Companion module: `commons_watch` — file watcher over the SSoT + trackers (`docs/guides/COMMONS_WATCH.md`).
- Integration: `docs/guides/WORKABLE_ITEMS_INTEGRATION.md` (the outbound/inbound flow this store feeds).
- Constitution: Helix §11.4.93 (SQLite SSoT) + §11.4.95 (DB tracked-in-git, canonical `docs/workable_items.db` path).

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on beyond the vendored `modernc.org/sqlite` already pinned in `commons_workable/go.mod`. All behavioural claims are grounded in the cited source files as of 2026-05-30.